

IoTpentBench: A Network-Scale Benchmark for LLM-Based IoT Penetration Testing

Anonymous Authors

Paper #XXXX, Anonymized for Double-Blind Review

Abstract—Autonomous LLM penetration-testing agents are typically evaluated on single hosts, yet IoT deployments expose risk through network reachability: gateways, brokers, cameras, and OT services form attack paths whose security depends on pivots across devices. We present IoTpentBench, a reproducible network-scale IoT benchmark with 15 Proxmox/Ansible scenarios, 145 devices in total (133 in vulnerability-injected scenarios), and 229 injected vulnerabilities mapped to OWASP IoT Top 10 and MITRE ATT&CK for ICS. We also introduce MESHSCOUT, a six-phase network-native harness that combines topology priors, active discovery, per-device triage, per-vulnerability verification, intrusion attempts, and report generation. The benchmark adds two metrics absent from single-host evaluations: evidence-level coverage (L1 detection, L2 exploitation, L3 exfiltration) and Multi-Hop Reach (MHR_k), which measures recall on vulnerabilities that require at least k pivots or zone transitions. We describe the benchmark, evaluator, and experimental protocol for isolating architectural effects independently from model choice.

Index Terms—IoT security, penetration testing, large language models, autonomous agents, benchmark, cyber-physical systems

1. Introduction

Modern IoT deployments are *networks*, not hosts: a smart building chains cameras, brokers, and controllers behind a gateway; an industrial site puts Modbus PLCs behind a jump host. The attack surface that matters is the union of reachability paths, and real-world incidents (Mirai [25], Verkada [26], Stuxnet [27]) all materialised as multi-hop pivots across that surface.

LLM-pentesting agents have matured rapidly and report substantial gains on public test corpora (Table 1), yet every such corpus is a collection of *isolated containers or VMs* and every agent is scored one target at a time. Three properties of a real IoT deployment remain invisible to that metric: *hop depth* (behind-pivot vulnerabilities), *IoT/OT protocols* (MQTT, Modbus, LoRaWAN, CoAP), and *evidence depth* (detection vs exploitation vs exfiltration). These gaps are properties of the evaluation surface, not of the agent: improving the agent alone does not close them.

Contributions.

- **IoTpentBench**: a reproducible network-scale benchmark for LLM pentest agents on IoT. It ships 15

scenarios (10 main covering 5 architectural patterns, 3 scalability stressors up to 35 devices, 2 hardened controls for False Positive rate), 145 devices in total, and 229 vulnerabilities injected via Ansible, with ground truth mapped to OWASP IoT Top 10 and MITRE ATT&CK for ICS.

- **MESHSCOUT**: a network-native harness whose six phases (topology prior, discovery, per-device triage, per-vulnerability verification, intrusion, report) keep LLM context tractable at $/24$ scale.
- **L1/L2/L3 evidence scoring and MHR_k** : metrics that separate detection, exploitation, and exfiltration, and that measure recall as a function of firewall-hop distance.
- **Evaluation protocol** holding the LLM constant across systems to isolate architectural contributions. The primary comparison uses MESHSCOUT, two internal ablations, and Nmap NSE; optional LLM-agent baselines are treated as adapters when their tool interfaces can be reproduced under the same model and budget.

All artifacts (Ansible playbooks, ground truth, evaluator, run logs) are released at ANONYMIZED_URL.

2. Related Work

This section reviews prior work relevant to our approach. We first survey LLM-driven penetration-testing agents and the benchmarks used to evaluate them, then discuss the classical attack-graph literature that informs MESHSCOUT’s topology-aware design, and finally review IoT firmware analysis and the security frameworks underlying our scenario taxonomy.

2.1. LLM Penetration-Testing Agents

Happe and Cito [1] were among the first to show that a single LLM driving a shell can autonomously perform privilege-escalation tasks on Linux VMs. PENTESTGPT [3] then introduced a multi-module design (three coordinated LLM sessions over a *Pentesting Task Tree*), evaluated on 182 hand-curated HackTheBox and VulnHub sub-tasks across 13 targets, reaching 66.5% subtask and 46.2% end-to-end completion against 52.2%/38.5% for plain GPT-4 and

23.1%/7.7% for GPT-3.5. HACKINGBUDDYGPT [2], AUTOATTACKER [4] and NEBULA [5] extend this single-agent line into open-source tools.

Later systems push specialisation from cognitive roles to penetration phases. VULNBOT [6] decomposes the workflow into three phases (reconnaissance, scanning, exploitation) driven by five cooperating modules over a *Penetration Task Graph*; it reports 69.1% subtask and 30.3% end-to-end completion on AutoPenBench with Llama 3.1-405B, a 21-point end-to-end gain over the same model used single-agent. AUTOPENTESTER [7] adds retrieval-augmented generation across five LLM agents and reports a +27% subtask gain over PENTESTGPT on HackTheBox. PENTESTAGENT [8] and the production-grade CAI [9] generalise this multi-agent blueprint with modular tool use and a public leaderboard.

A complementary thread improves the underlying agent rather than its orchestration. PENTEST-R1 [10] applies two-stage reinforcement learning on reasoning traces, and XOFFENSE [11] LoRA-fine-tunes Qwen3-32B on offensive write-ups (TryHackMe, HackTheBox, VulnHub, WHITERABBITNEO), reaching 79.17% subtask completion on AutoPenBench and surpassing PENTESTGPT and VULNBOT. CHECKMATE [16] couples an LLM agent with a *Classical Planning+* module encoding preconditions and effects of predefined attack actions, improving success rates by more than 20% over Claude Code + Sonnet 4.5 on the Vulnhub benchmark.

Despite the diversity of these designs, published scores are usually reported at the level of a single target or challenge (a HackTheBox machine, a VulnHub VM, an AutoPenBench task, or a CTF environment). Some environments require multi-step exploitation, but the reported metrics do not expose subnet-level reachability, hop depth, or per-vulnerability progress across an IoT/OT topology with protocols such as MQTT, Modbus, LoRaWAN, and CoAP.

2.2. LLM Cybersecurity Benchmarks

Challenge-level corpora. AUTOPEBENCH [17] is the dominant evaluation substrate — 33 Docker-container tasks (22 in-vitro, 11 real-world CVEs) with milestone and flag-based scoring — used by VULNBOT, PENTEST-R1, and XOFFENSE. VULHUB [24] fills a related role as a large Docker-Compose collection of reproducible vulnerable environments, used by CHECKMATE. CAIBENCH [21] is the broadest academic effort, spanning CTFs, attack-and-defense ranges, and knowledge tests; its RCTF2 subset adds 27 robotics challenges — the most explicit cyber-physical inclusion in this family. Their public scoring is challenge- or flag-level: they do not provide a per-vulnerability ground truth tied to network reachability, hop depth, or IoT/OT application protocols (MQTT, Modbus, LoRaWAN). Table 1 summarises each system’s scope and scoring scheme.

Multi-host labs. LUDUS [22] and the GOAD [23] Active-Directory scenario deliver Proxmox+Ansible deployments

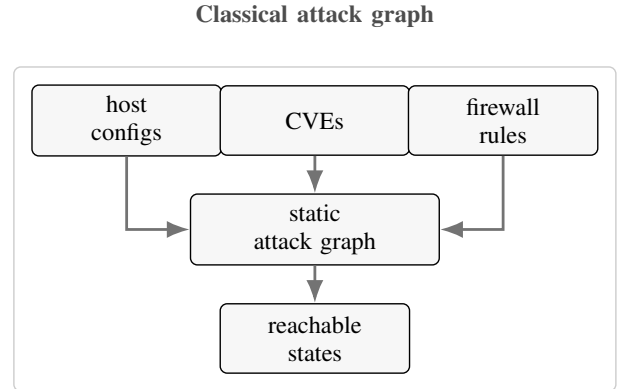


Figure 1. Classical attack-graph analysis computes reachable attacker states from a static model of hosts, vulnerabilities, and firewall rules.

of representative enterprise networks, the closest methodological precedent to our infrastructure. They do model multi-host compromise and lateral movement, so the gap is not the existence of multi-host cyber ranges. Rather, these labs are not packaged as LLM-agent benchmarks: they do not ship a per-vulnerability ground-truth schema, hop-depth annotations, or a scoring procedure, and they target enterprise AD rather than IoT/OT protocols. We found no prior benchmark that combines multi-host reachability, IoT/OT protocols, per-vulnerability machine-readable ground truth, and evidence-level scoring beyond binary flag capture (Table 1).

2.3. Attack Graphs and Network Reachability

Representing a network as a graph for security analysis predates deep learning by decades. Phillips and Swiler [28] introduced graph-based network-vulnerability analysis, Sheyner *et al.* [29] developed automated attack-graph generation via model checking, and MULVAL [30] cast the problem as Datalog inference, becoming the de facto reference for attack-graph research. These systems compute reachable attacker states from a *static* encoding of host configurations and exposed services. We borrow their reachability abstraction (nodes are devices, edges are firewall-permitted protocol flows) but MESHSCOUT operates on a *live* network: edges are validated by actual probes, the LLM proposes the next action, and the graph is updated as new hosts are discovered. Classical attack-graph generation is therefore complementary rather than competing. Figures 1 and 2 summarize this distinction.

2.4. IoT Security Context

Scenario design and ground-truth taxonomy follow three references: the OWASP IoT Top 10 [31] for vulnerability categories, MITRE ATT&CK for ICS [32] for OT-tactic annotations, and NIST SP800-82 Rev. 3 [33] for OT security and segmentation conventions, notably the IT/OT split underpinning pattern P3 (§4.2).

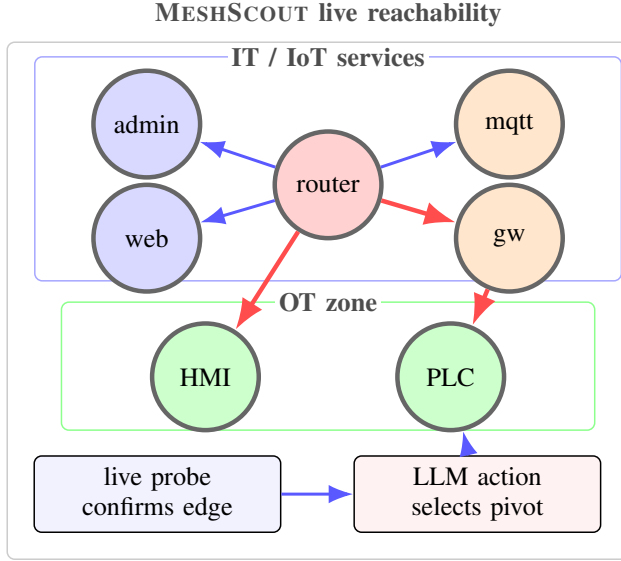


Figure 2. MESHSCOUT keeps the reachability abstraction but updates the graph during execution from live probes and LLM-selected actions. Blue edges denote validated reachability; red edges denote pivot-dependent paths into deeper IoT/OT zones.

Empirical IoT-security artefacts fall into two adjacent families. First, network-traffic datasets such as IoT-23 [34], TON_IoT [35], and Edge-IIoTset [36] provide labeled traces for intrusion-detection and anomaly-detection research. They are valuable for defensive classifiers, but they are not deployable pentest targets with per-vulnerability ground truth, exploit evidence, or controllable reachability. Second, firmware-focused work studies vulnerable device images: Costin *et al.* [37] performed a large public firmware study, FIRMADYNE [38] introduced dynamic firmware emulation for vulnerability discovery, and FIRMAE [39] substantially improved emulation success rates. OWASP IOT-GOAT [40] provides a deliberately insecure OpenWrt-based firmware image used as a teaching artefact. These datasets and firmware images are complementary to IoTPentBench: they exercise traffic classification or single-device firmware analysis, while IoTPentBench evaluates LLM agents on deployed multi-device networks with firewall-enforced reachability.

3. Threat Model and Problem Definition

Problem statement. Given a target subnet with a designated *entry point* e (the attacker’s initial foothold, declared in each scenario’s topology YAML, §4.4), *network-scale penetration testing* is the task of discovering all hosts reachable from e , identifying their vulnerabilities, producing evidence of exploitability, and reporting findings aggregated across the full network. The *unit of evaluation* is the network, not an individual host: IoT risk is driven by pivot chains (e.g., entry point $\rightarrow$ gateway $\rightarrow$ MQTT broker $\rightarrow$ OT controller), and single-host metrics are blind to vulnerabilities that only become reachable after one or more pivots.

TABLE 1. GAP ANALYSIS. *Scale*: SINGLE HOST VS. MULTI-DEVICE NETWORK. *IoT/OT*: APPLICATION-LAYER IoT/OT PROTOCOLS ON THE TEST SURFACE. *GT*: GROUND-TRUTH GRANULARITY (FLAG, TASK-STEP, PER-VULNERABILITY, NONE). *MH*: MULTI-HOP SCORING.

System	Scale	IoT/OT	GT	MH
<i>LLM pentest agents</i>				
Happe & Cito [1]	1 host	✗	flag	✗
PentestGPT [3]	1 host	✗	task-step	✗
VulnBot [6]	1 host	✗	task-step	✗
AutoPentester [7]	1 host	✗	task-step	✗
CAI [9]	1 host	✗	flag	✗
Pentest-R1 [10]	1 host	✗	task-step	✗
xOffense [11]	1 host	✗	task-step	✗
EnlGMA [12]	1 host	✗	flag	✗
CheckMate [16]	1 host	✗	task-step	✗
<i>Evaluation corpora</i>				
AutoPenBench [17]	1 host	✗	flag	✗
CAIBench / RCTF2 [21]	1 host	partial	flag	✗
Vulhub [24]	1 host	✗	flag	✗
Ludus + GOAD [22], [23]	network	✗	none	✗
IoTpentBench (ours)	network	✓	per-vuln	✓

Attacker model. The attacker is an external adversary who has gained Layer-3 access to the target subnet (for example, by exploiting a perimeter service or via physical LAN access) but holds *no* prior credentials, no out-of-band knowledge of deployed software, and no privileged vantage beyond what active scanning reveals. Their objective is data exfiltration (sensor readings, credentials) or OT disruption (reaching industrial-control services behind a segmentation boundary). They may use standard pentest tooling (Nmap, MQTT clients, SSH, HTTP) augmented by an LLM with tool-calling, within a bounded turn budget (§5.9).

Success criterion and evidence levels. A finding is *solved* at the deepest evidence level the agent can demonstrate within its turn budget. We distinguish three levels: L1 *detection*: a port scan or banner confirms the issue (e.g., unauthenticated MQTT port 1883 open); L2 *exploitation*: an active interaction demonstrates it (e.g., anonymous MQTT subscribe on #, SSH login with default credentials); L3 *exfiltration*: sensitive content is retrieved (e.g., MariaDB root dump, camera RTSP stream). This ordering defines both what Phase 4 of MESHSCOUT is prompted to achieve (§5.5) and how findings are scored (§4.5).

Scope. We evaluate at the IP/application layer on virtual IoT networks. Out of scope: physical and side-channel attacks, RF-layer exploitation requiring SDR hardware, firmware extraction, post-exploitation persistence, and insider threats. These remain future work (§10).

4. IoTpentBench: A Network-Scale Benchmark

4.1. Design principles

IoTpentBench is built around six design principles derived from the gap analysis in §2:

- **Reproducibility.** The entire infrastructure deploys from Ansible playbooks against a Proxmox cluster;

every vulnerability is injected idempotently with no manual post-deployment steps.

- **Architectural diversity.** Scenarios span five network patterns (§3) with distinct attacker reachability graphs encoded in the topology metadata and, for segmented/VLAN cases, enforced by OpenWrt firewall rules rather than service labels alone.
- **Protocol coverage.** Each scenario mixes multiple IoT and OT protocols (MQTT, Modbus, LoRaWAN gateways, CoAP, SNMP, plus standard SSH/HTTP/FTP/Telnet) to exercise protocol-specific reasoning.
- **Ground-truth granularity.** Every injected vulnerability is documented in YAML with device, IP, role, severity, category, OWASP IoT and MITRE ICS mappings, optional CVE, indicators (expected detection strings), and a verification command.
- **Scalability axis.** Three scenarios (s_11, s_12, s_13) stress network size at 15, 17, and 35 devices respectively, while the 10 main scenarios stay in the 4–8 device range. This separates the pattern-coverage axis from the size axis.
- **Hardened controls.** Two scenarios (s_1h, s_4h) reuse the topologies of s_1 and s_4 with all injected vulnerabilities removed. They measure agent False Positive rate on a deployment that should return “no findings”.

4.2. Five architectural patterns

Concretely:

- 1) **P1 — Flat.** All devices are directly reachable from the entry point; no pivot is required.
- 2) **P2 — Gateway-centric.** A single IoT gateway mediates all access to back-end services; inter-service traffic is blocked without gateway compromise.
- 3) **P3 — Segmented IT/OT.** Three zones (admin/IT/OT) with access-control lists; the OT zone (Modbus PLCs, HMI) is reachable only via a jump host.
- 4) **P4 — Hub-centric star.** A central home-automation hub is the only device that can reach MQTT broker, DB, and camera; peripherals cannot talk to each other directly.
- 5) **P5 — Edge-cloud pivot.** Edge subnet and cloud subnet are separated; only the edge gateway bridges them, making cloud-side services unreachable without edge compromise.

IoTpentBench ships 15 scenarios organised on three axes: 10 *main* scenarios that instantiate the 5 architectural patterns above (with small-to-medium device counts, 4–8 devices each), 3 *scalability* scenarios that stress network size (15, 17, and 35 devices) using logical multi-zone and real-VLAN variants, and 2 *hardened controls* that ship identical topologies to s_1 and s_4 *without* any injected vulnerability, used to measure agent False Positive rate. The full mapping is given in Table 2.

TABLE 2. THE 13 VULNERABILITY-INJECTED SCENARIOS OF IOTPENTBENCH (133 DEVICES, 229 INJECTED VULNERABILITIES). THE 2 HARDENED CONTROLS ADD 12 DEVICES AND 0 VULNERABILITIES, FOR 15 SCENARIOS AND 145 DEVICES IN TOTAL. *Main* SCENARIOS COVER THE 5 ARCHITECTURAL PATTERNS; *Scalability* SCENARIOS STRESS NETWORK SIZE.

ID	Pattern / role	Dev.	Vulns	Theme
<i>Main scenarios (pattern coverage)</i>				
s_1	P1 Flat	4	12	Minimal IoT (MQTT/Web/SSH)
s_2	P1 Flat	6	13	Flat IoT + IT mix
s_3	P2 Gateway-centric	8	18	NATO lab replica
s_4	P3 Segmented IT/OT	8	18	Industrial (Modbus/HMI)
s_5	P4 Hub-centric star	8	15	Smart building (cameras/NVR)
s_6	P4 Hub-centric star	6	16	Home automation hub
s_7	P5 Edge-cloud pivot	6	14	Edge → cloud API
s_8	P3 Segmented IT/OT	8	14	IT/IoT/OT 3-zone industrial
s_9	P1 Flat (mesh)	6	11	Mesh IoT smart-city outdoor
s_10	P1 Flat (variants)	6	13	Role-variant stress
<i>Scalability stressors</i>				
s_11	Smart City (flat)	15	23	3 zones same /24 (no isolation)
s_13	VLAN-segmented	17	20	Real Proxmox VLAN bridge (3 VLANs)
s_12	Smart City (large)	35	42	Largest network (34 LXC services)
Total		133	229	

TABLE 3. VULNERABILITY CATEGORIES AND SEVERITY DISTRIBUTION ACROSS ALL 13 VULNERABILITY-INJECTED SCENARIOS OF IOTPENTBENCH (229 INJECTED INSTANCES; THE 2 HARDENED CONTROLS CONTRIBUTE 0 BY DESIGN). COUNTS DERIVED FROM GROUND_TRUTH/SCENARIO_*.YAML.

Category	Count	CRIT	HIGH	MED	LOW
misconfiguration	68	7	36	25	0
no_authentication	41	17	21	3	0
data_exposure	38	0	14	24	0
default_credentials	29	3	26	0	0
info_disclosure	13	0	0	0	13
code_injection	11	10	1	0	0
cve	9	0	3	6	0
insecure_update	9	0	4	5	0
weak_crypto	6	0	0	1	5
missing_header	3	0	0	0	3
privilege_escalation	2	0	2	0	0
Total	229	37	107	64	21

4.3. Deterministic vulnerability injection

Vulnerabilities are injected by a single Ansible role (04_inject_vulns.yml) parameterized by scenario_id. Each scenario YAML lists router_vulns (OpenWrt-level: Telnet, WAN admin, FTP) and per-service role definitions that trigger role-specific injections: *e.g.*, the mqtt_broker role sets allow_anonymous true and binds on 0.0.0.0:1883; the ssh_server role installs weak credentials (admin:admin) and enables PasswordAuthentication; the db_server role starts MariaDB with a passwordless root. Category distribution is shown in §3.

4.4. Ground-truth schema

Each scenario ships a YAML file describing every injected vulnerability. An entry example (from

Draft schematic: five topology diagrams side-by-side.
 (Flat) — (Gateway-centric) — (Segmented IT/OT) — (Hub-star) — (Edge-cloud)

Figure 3. The five architectural patterns in IoTpentBench. Reachability is encoded in the scenario topology and enforced by OpenWrt firewall rules in the segmented/VLAN cases.

```
- id: V1
  device: s1-mqtt
  ip: "192.168.100.11"
  role: mqtt_broker
  title: "MQTT without authentication"
  severity: high
  category: misconfiguration
  owasp_iot: "I1 / I9"
  mitre_ics: "Initial Access, Collection"
  cve: null
  indicators:
    - "allow_anonymous true"
    - "Anonymous clients connected"
  verification: "mosquitto_sub -h 192.168.100.11 -t '#' -v"
```

Figure 4. Ground-truth entry example.

scenario_1.yaml):

Each scenario YAML may additionally declare a top-level `bonus_types` list of vulnerability types that are *adjacent* to the injected set (e.g., a hardening recommendation related to the same service) and which, if reported by an agent, are neither counted as True Positives nor as False Positives. This avoids penalising agents for legitimate adjacent findings while keeping the recall denominator fixed. The evaluator leniency this introduces is discussed in §8 (Limitations).

4.5. Evaluator

The evaluator (§5.7 integrates results into Phase 6) matches each LLM finding against ground truth in three ordered passes, with each pass consumed at most once per GT entry: (1) exact CVE match; (2) (ip, vuln_type) with type canonicalised through the taxonomy (§5.8); (3) a last-resort (ip, severity) fallback for findings whose type string is unknown but whose affected host and impact match a GT entry. Findings that match none of the three passes are counted as False Positives. Severity weights are $w = \{\text{critical}=4, \text{high}=3, \text{medium}=2, \text{low}=1\}$; loose fallback matches (pass 3) apply $0.5\times$, severity mismatches within an otherwise valid match apply $0.75\times$. The composite score is the weighted True-Positive sum normalized by $\sum_{g \in \text{GT}} w(g)$.

Severity-aware dedup. Before evaluation, the Phase 3 aggregation step canonicalises vulnerability types, removes known noise classes, and deduplicates collisions on (ip, type, port) by retaining the *lower* reported severity. This conservative rule counteracts the severity inflation we observed in pilot runs and prevents duplicate findings from increasing match scores.

Evidence levels. Beyond match accuracy, the evaluator tracks, per True Positive, the deepest *evidence level* achieved by the harness: L1 (detection, banner or port),

L2 (exploitation, authenticated action), L3 (data exfiltration, sensitive content retrieved). We report L_k -coverage = $|\{t \in \text{TP} : \text{level}(t) \geq k\}|/|\text{TP}|$. This metric cannot be computed on single-host binary-flag benchmarks and captures a dimension of pentest quality that existing LLM-pentest evaluations conflate.

Hop depth and Multi-Hop Reach. Let $G = (V, E)$ be the firewall-permitted reachability graph (edges are firewall-allowed protocol flows from §5.2) and let $e \in V$ be the attacker entry point (the subnet ingress). For each ground-truth vulnerability g attached to device $d_g \in V$, we define its *hop depth* as the number of required pivots or zone transitions from the entry point. Directly reachable vulnerabilities have $\text{hop_depth} = 0$:

$$\text{hop_depth}(g) = \min_{p \in \text{paths}(e \rightarrow d_g)} \max(0, |p|_E - 1),$$

where $|p|_E$ is the number of edges in the path. Thus a device one firewall edge away from the entry point has depth 0, and each additional required pivot or zone transition increments the depth. A flat scenario has $\text{hop_depth}(g) = 0$ for every directly reachable device; a gateway-centric scenario places back-end services at $\text{hop_depth} \geq 1$; an IT/OT-segmented scenario places OT services at $\text{hop_depth} \geq 2$. We then define the Multi-Hop Reach at depth k as the recall restricted to deep ground-truth findings:

$$\text{MHR}_k = \frac{|\{g \in \text{GT} : \text{hop_depth}(g) \geq k \wedge g \in \text{TP}\}|}{|\{g \in \text{GT} : \text{hop_depth}(g) \geq k\}|}.$$

Raw recall is reported separately as Task Completion; MHR_1 , MHR_2 , and MHR_3 measure progressively deeper behind-pivot findings. A single-host agent run on only directly exposed hosts is expected to collapse on MHR_k for $k \geq 1$ unless it can establish and use a new vantage point.

5. MESHSCOUT: A Network-Native LLM Pentest Harness

5.1. Overview

The harness decomposes network pentest into six phases with typed deliverables. The key design decision is *parallelization at the device and vulnerability granularity*: rather than a single monolithic agent that must hold the full network state, we spawn dedicated micro-agents per target with constrained tool sets, then aggregate deterministically. This keeps each micro-agent’s context bounded *per device* at $O(1)$ in the number of network devices, with cross-device state only re-introduced in Phase 5 (intrusion) and Phase 6 (report).

Draft schematic: 6-phase pipeline diagram.
Phase 1 (topology prior) → Phase 2 (discovery) → Phase 3 (per-device triage) → Phase 4 (per-vuln verification) → Phase 5 (intrusion) → Phase 6 (network report).

Figure 5. MESHSOUT pipeline. Solid arrows are deliverables; dashed boxes indicate LLM agent calls; Phase 3 and Phase 4 spawn micro-agents in parallel (§5.4–§5.5).

*Draft schematic: Phase 3 parallelization — n devices
→ n LLM agents in parallel → deterministic merge.*

Figure 6. Per-device parallelization. Each micro-agent receives only its device’s scan output, avoiding the context explosion of a monolithic network-level agent.

5.2. Phase 1: Topology prior

Phase 1 loads a graph model of the target network as a directed graph $G = (V, E)$ where V are devices and E are links labeled by protocol. Two modes are supported: (i) *scenario mode* loads the predefined topology from the benchmark YAML; (ii) *discovery mode* starts from an empty graph and instructs Phase 2 to populate V via network scans. The output is a markdown deliverable enumerating attack surface and preliminary pivot candidates (betweenness-centrality ranked).

5.3. Phase 2: Full-network discovery

Phase 2 executes active reconnaissance across the target /24: `nmap_discovery` (host discovery), `nmap` (service/version), `arp_scan` (L2 + vendor), and protocol-specific probes (`mqtt_listen` subscribing to # on discovered brokers, `ssh_audit` against SSH servers, `curl_headers` on HTTP). Unlike single-host harnesses, the agent is not given a target IP: it must derive one from the CIDR scope.

5.4. Phase 3: Per-device parallel triage

Phase 3 runs in three sub-phases:

- 1) **Deterministic scanner pass.** Ansible-driven subprocess wrappers (`ssh_audit`, `curl_headers`, `mqtt_listen`, `modbus_scan`, ...) run in parallel per discovered device and dump structured scan results to disk.
- 2) **Per-device LLM triage.** For each device, a dedicated LLM agent receives only that device’s Phase 2+3a outputs and has access to a minimal tool set: `http_get` (follow-up file retrieval) and `cve_search` (CPE-to-CVE lookup via NVD). The agent produces a JSON vuln list scoped to that device.
- 3) **Deterministic merge.** Per-device outputs are concatenated, canonicalized through the taxonomy (§5.8), and deduplicated via severity-aware rules.

This decomposition is essential for network scale: a monolithic agent on an 8-device scenario already needs to

fit ~15–40k tokens of scan output into a single context and reason about cross-device relationships simultaneously, and the cost grows linearly with n . Per-device micro-agents flatten this to a constant size, at the cost of losing some cross-device inference capability, which is recovered in Phase 5.

5.5. Phase 4: Per-vulnerability verification with evidence levels

Phase 4 spawns one micro-agent per Phase 3 finding (in parallel, with a concurrency cap). Each micro-agent receives the finding (IP, type, port, details) and full operational tool access: `ssh_login`, `mysql_query`, `mqtt_listen`, `http_get`, `ftp_list`, `modbus_scan`, `telnet_connect`. The agent is prompted to produce the deepest evidence level it can (L1–L3, §4.5) within a bounded turn budget. Outputs include: `status` (CONFIRMED, NOT_EXPLOITABLE, ERROR), `evidence_level`, `command` executed, `stdout` excerpts, and `data_extracted` (when L3).

Findings in CONFIRMED state from Phase 3 override Phase 4 ERROR/FAILED statuses (rationale: Phase 3 is hypothesis generation with directly-observed indicators; Phase 4 crashes should not erase directly-observed findings).

5.6. Phase 5: Intrusion and lateral movement

Phase 5 turns the per-device vulnerability inventory from Phases 3–4 into a network-scoped infiltration campaign. Given the infrastructure graph (Phase 1) and the confirmed credentials/exposed services collected so far, an intrusion agent attempts: (i) *credential spraying* across discovered hosts (reusing harvested SSH/HTTP/MQTT/DB credentials), (ii) *lateral movement* along firewall-permitted edges, with each successful hop adding a new vantage point to the graph, and (iii) *crown-jewel access* on back-end services (databases, NVRs, historian, cloud APIs) that are unreachable from the entry point in segmented patterns (§4.2). The agent records, for every hop, the source node, destination node, edge protocol, and credentials/exploit used. This phase is the sole producer of L_3 evidence and of MHR_k findings at $k \geq 2$; ablating it (Tab. 4, 6) forces those columns to zero by construction.

5.7. Phase 6: Network-level report

Phase 6 consumes all prior deliverables and produces a network-scoped report with: device inventory, per-device vulnerabilities with severity, attack surface summary, multi-hop intrusion narrative (from Phase 5), and actionable recommendations ordered by severity $\times$ exposure. This is the

only phase with cross-device aggregation reasoning, deliberately placed at the end to avoid bloating earlier phases.

5.8. Taxonomy and tool abstraction

A single module resolves LLM-produced vulnerability type strings into a canonical taxonomy of 11 categories (§3), filtering noise (e.g., meta-observations) and CONFIG-only findings. Tools are defined declaratively as YAML (schema: name, description, JSON-schema parameters, subprocess command template), so adding a new protocol probe requires no Python code.

5.9. Complexity and cost envelope

For a network with n devices and an average of v findings per device, the harness issues $1(P1) + 1(P2) + n(P3b) + nv(P4) + 1(P5) + 1(P6) = O(nv)$ LLM agent invocations, each bounded by a fixed turn budget (50 for Phase 1, 50 for Phase 2, 10 per device for Phase 3b, 10 per finding for Phase 4, 30 for Phase 5, 20 for Phase 6). One *call* is one round of (*prompt* $\rightarrow$ *tool calls* $\rightarrow$ *response*); one *turn* is one such round inside the budget. Total cost scales linearly with network size, not combinatorially.

6. Experimental Setup

Infrastructure. All scenarios run on a single Proxmox VE host. Networks are Linux bridges (vbr1, 192.168.100.0/24) with an OpenWrt router as gateway. Per-scenario deployment and teardown are automated via Ansible and complete in under 10 minutes per scenario.

LLM model: fairness lock. To isolate *architectural* contributions from model-specific advantages, all LLM-driven systems compared in this work, including our pipeline (and its ablations) and all LLM baselines, run on the same general-purpose model: **MiniMax-M2.7** [41]. We deliberately avoid a multi-model matrix: a fair comparison of architectures requires the model variable to be held constant. CAI’s specialised `alias1` model (security-fine-tuned, paywalled) is intentionally not used: this study addresses the architecture axis, not the model fine-tuning axis. The choice of MiniMax-M2.7 also keeps out-of-pocket cost bounded under our research subscription, enabling the full $k=3$ -run variance estimation across systems and scenarios without budget trade-offs.

Baselines. The core evaluation compares MESHSCOUT against two internal ablations and one non-LLM scanner baseline. External LLM pentest frameworks are discussed as related work; we only include them in quantitative tables when their tool interface can be adapted to the same model, scenario scope, and turn budget without changing their architecture.

- **MESHSCOUT-w/o-Phase 1** disables the topology prior (`--phases 2 3 4 5 6`), testing the value of infrastructure-graph guidance.

- **MESHSCOUT-w/o-Phase 5** disables the intrusion phase (`--phases 1 2 3 4 6`), testing the value of structured multi-hop lateral movement.
- **Nmap NSE** runs Nmap with all NSE scripts (`--script=default,vuln,auth`) against the same target CIDR. Scanner outputs are converted to the evaluator’s JSON schema.

Internal ablations. The two ablations above isolate our key design choices: topology-guided planning and a dedicated lateral-movement phase. Phase 5 is the only phase that intentionally establishes new vantage points, so removing it should primarily affect L_3 evidence and MHR_k for $k \geq 1$ (§5.6).

Variance. Each (system, scenario) cell is run $k=3$ times. We report mean, standard deviation, and 95% bootstrap confidence intervals. Pairwise differences (our pipeline vs each baseline) are tested with Mann-Whitney U on F1 and weighted Score per scenario. *Power caveat:* with $n_1 = n_2 = 3$, the two-sided Mann-Whitney U test has a minimum reachable p -value of 0.10, so individual per-scenario comparisons are indicative rather than conclusive. We mitigate this by aggregating across the ten main scenarios (combined $n = 30$ per system) for the headline cross-system tests, where power is sufficient. This protocol still exceeds the single-run reporting of PentestGPT [3], AutoPentester [7], AutoPenBench [17], and CAI [9], and matches the multi-run averaging of VulnBot [6] (5-experiment) and Pentest-R1 [10] (pass@3).

Metrics. We report five families of metrics, chosen to (i) align with prior work’s expected vocabulary and (ii) surface architectural differences invisible to host-level benchmarks:

- **Task completion:** Recall against the ground-truth vulnerability set, the network-scale analogue of the sub-task / task completion rate reported by PentestGPT, VulnBot, AutoPentester, Pentest-R1, and xOffense.
- **Quality:** Precision, F1, and weighted Score (CVSS-severity-weighted Recall, §4.5). Score plays the role of AutoPentester’s “vulnerability coverage” but with severity weights.
- **Multi-Hop Reach:** MHR_k for $k \in \{1, 2, 3\}$, the fraction of ground-truth vulnerabilities at `hop_depth` $\geq k$ that are reached. Existing LLM-pentest benchmarks generally report target, sub-task, or flag completion rather than recall as a function of network depth.
- **Evidence depth:** L_1 (detection), L_2 (exploitation), L_3 (exfiltration) coverage; absent from binary flag-capture benchmarks.
- **Cost envelope:** prompt+completion tokens and monetary cost (USD) from provider billing APIs, plus wall-clock time per scenario, matching CAI [9], AutoPentester, and Pentest-R1.

Compute and budget. All runs are bounded by per-phase turn caps and the concurrency limit in Phase 4.

Draft chart: bar chart, F1 and MHR₂ for MESHSCOUT, ablations, and Nmap NSE.

Figure 7. Baseline comparison on IoTpentBench. Scanners capture known CVEs but miss configuration vulnerabilities and do not establish new vantage points. The Phase 5 ablation isolates the value of explicit lateral movement on multi-hop reach (MHR₁–MHR₃).

Draft chart: heatmap scenario $\times$ model, cell = weighted Score.

Figure 8. Per-scenario Score heatmap. [TODO: Note which pattern consistently breaks LLMs.]

We report prompt/completion tokens, token-estimated USD cost, and actual out-of-pocket cost separately, because subscription-backed providers can make the latter lower than per-token estimates.

7. Evaluation

7.1. Overall performance

[TODO: Write §7.1 prose after final runs. Report headline F1/Score, MHR₁–MHR₃, and whether Phase 1/Phase 5 ablations explain the gap.]

7.2. Cross-benchmark comparison on single-host corpora

The Vulhub run in Table 5 is deliberately marked as preliminary. The current snapshot covers only the first 20 cases completed or failed across six workers, of which 15 reached the agent and 5 failed during environment setup before any model call. Because execution order follows each worker shard rather than a random permutation, this sample is not representative of the 191-case corpus. The 15 completed agent runs consumed 16.66M tokens and \$5.24 in token-priced costs, or about 1.11M tokens and \$0.35 per case.

The snapshot is nevertheless useful for calibrating the evaluation protocol. Vulhub does not provide a uniform flag oracle, so an agent can demonstrate code execution or file disclosure and still self-report failure while searching for a nonexistent flag. Re-reading the generated proof.json files changes the observed success rate from the automatic 2/15 (13%) to 3/13 (23%) confirmed exploits after excluding two non-deployable targets; four additional cases show partial exploitation without a final flag. We therefore use this row only as a single-host sanity check and will replace it with the full-run oracle result once all 191 cases have completed.

7.3. Per-architecture analysis

[TODO: Write §7.3 — pattern-by-pattern commentary. Expected finding: gateway-centric and hub-star degrade most, because they require pivot reasoning. Edge-cloud may also degrade on cloud-side service discovery.]

Draft chart: Pareto plot, USD cost vs weighted Score, one point per system (all on MiniMax-M2.7).

Figure 9. Cost–performance Pareto across systems on MiniMax-M2.7; the multi-model frontier is deferred to future work (§10). [TODO: Annotate which system dominates on each axis and the efficient frontier.]

7.4. Evidence-level depth

[TODO: Write §7.4 — expected: detection (L1) near-parity with scanners, but L2/L3 is LLM-only. This is the strongest argument for LLM agents in this context.]

7.5. Per-protocol analysis

[TODO: Write §7.5 — 2 paragraphs. Expected: LLMs excel on MQTT/SSH/HTTP (text-rich, well-represented in training), fall short on Modbus/LoRaWAN (binary, sparse training data). Report numbers per protocol.]

7.6. Failure modes and cost/time envelope

[TODO: Write §7.6 — categorize failure modes: (a) hallucinated findings on hardened controls, (b) missed vulnerabilities on binary protocols, (c) over-reporting info-disclosure without severity calibration. Include a brief quantitative table.]

8. Discussion and Limitations

Where LLMs help, where they don’t. IoTpentBench is designed to separate several failure modes that are usually collapsed into one binary success signal. In the final analysis we expect this separation to show where LLM agents help most (text-rich misconfigurations, default credentials, and information disclosure) and where they require additional support (binary industrial protocols and cross-device inferences that require a persistent network view). The intended deployment model is therefore hybrid: LLM agents for configuration auditing, traditional scanners for CVE coverage, and human operators or specialised tooling for the OT/ICS layer.

Implications for agentic systems. The harness’s per-device and per-vulnerability micro-agent structure is one answer to context scaling, but it cedes cross-device inference. Designs that keep a global aggregator agent in the loop without paying the full network-context cost are an open research direction.

Limitations. (i) *Simulated infrastructure*. IoTpentBench scenarios emulate IoT devices on Linux VMs; real constrained hardware introduces embedded-specific attack surfaces (bootloaders, RTOS, firmware) we do not cover. (ii) *Evaluator leniency*. Bonus categories are accepted as non-FP even without ground-truth matches, which inflates precision. (iii) *Model variance*. With $k=3$ runs per configuration, confidence intervals on F1 remain wide for some

TABLE 4. RESULTS ON IOTPENTBENCH (10 MAIN SCENARIOS $\times$ 3 RUNS, MINIMAX-M2.7). TC = RECALL; SCORE = CVSS-WEIGHTED RECALL; MHR_k = RECALL AT HOP DEPTH $\geq k$; 0.00 = ZERO BY CONSTRUCTION (NO LATERAL MOVEMENT); “—” = METRIC NOT APPLICABLE.

System	Quality				Multi-Hop Reach			Cost	
	TC	Prec.	F1	Score	MHR ₁	MHR ₂	MHR ₃	Tokens (M)	USD
MESHSCOUT (full)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
MESHSCOUT w/o Phase 1 (no infra graph)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
MESHSCOUT w/o Phase 5 (no intrusion)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	0.00	0.00	0.00	[TODO:]	[TODO:]
Nmap NSE (non-LLM scanner)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	0.00	0.00	0.00	—	—

Mean over $k=3$ runs; 95% bootstrap CI; pairwise Mann–Whitney U vs each baseline.

TABLE 5. CROSS-BENCHMARK COMPARISON. MESHSCOUT (MINIMAX-M2.7) IS RUN ON THE SINGLE-HOST CORPORA USED BY PRIOR WORK; THEIR PUBLISHED NUMBERS ARE SHOWN FOR REFERENCE. MODELS DIFFER ACROSS ROWS — THIS TABLE IS NOT A FAIR MODEL COMPARISON BUT VERIFIES THAT MESHSCOUT IS COMPETITIVE ON ESTABLISHED SINGLE-HOST CORPORA WHILE ADDING THE NETWORK-SCALE DIMENSION. *E2E* %: FRACTION OF TASKS FULLY SOLVED. CHECKMATE REPORTS ONLY A RELATIVE GAIN OVER ITS BASELINE (ABSOLUTE NOT PUBLISHED). “N/R” = NOT REPORTED. * = PRELIMINARY PARTIAL RUN (SEE TEXT).

System	Model	Corpus	Sub-task %	E2E %
<i>AutoPenBench (33 tasks)</i>				
Llama 3.1-405B baseline [6]	Llama 3.1-405B	AutoPenBench	49.1	9.1
GPT-4o baseline [6]	GPT-4o	AutoPenBench	n/r	21.2
VulnBot [6]	Llama 3.1-405B	AutoPenBench	69.1	30.3
xOffense [11]	Qwen3-32B (LoRA)	AutoPenBench	79.2	n/r
MESHSCOUT (ours)	MiniMax-M2.7	AutoPenBench	[TODO:]	[TODO:]
<i>Vulhub (191 CVE tasks)</i>				
Claude Code (baseline) [16]	Sonnet 4.5	Vulhub	n/r	n/r
CheckMate [16]	Sonnet 4.5 + Plan+	Vulhub	n/r	+20%^{rel}
MESHSCOUT (ours)	MiniMax-M2.7	Vulhub	n/r	23%*

^{rel} Relative gain over Claude Code baseline; absolute not published by CheckMate. * Partial snapshot: 20/191 Vulhub cases extracted while the distributed run was still in progress; 15 reached the agent, 5 failed before invocation. The reported rate is manual proof-oracle success on 13 deployable cases (3 confirmed exploits); the automatic verdict gives 2/15 = 13%.

TABLE 6. EVIDENCE-LEVEL COVERAGE. L_k IS THE FRACTION OF TRUE POSITIVES REACHING LEVEL k OR DEEPER. ABSENT FROM BINARY-FLAG BENCHMARKS.

System	L_1 (detect)	L_2 (exploit)	L_3 (exfiltrate)
MESHSCOUT (full)	[TODO:]	[TODO:]	[TODO:]
MESHSCOUT w/o Phase 5	[TODO:]	[TODO:]	0.00
Nmap NSE	—	—	—

scenarios. (iv) *Prompt sensitivity*. We fix a single prompt family across models; per-model prompt tuning would likely raise scores. (v) *Static architectures*. The five patterns are fixed; real deployments evolve over time and include devices not represented here (RFID badge readers, industrial PLCs, medical devices).

9. Ethical Considerations

Isolation. All experiments run on an isolated Proxmox cluster with a dedicated bridge (vmbri1) and no route to external networks; the OpenWrt gateway’s WAN interface is firewall-isolated. No live hosts, production services, or third-party infrastructure are probed.

Responsible disclosure. IoTPentBench vulnerabilities are deterministically injected into our own lab VMs via

Ansible. No 0-days are introduced; all categories (misconfig, weak credentials, Terrapin CVE-2023-48795, outdated Dropbear) correspond to patterns already publicly documented. No responsible-disclosure process is triggered.

Misuse risk of released artifact. The benchmark artifact (Ansible playbooks, ground truth, evaluator) does not embed exploitation payloads for unknown vulnerabilities, and the injected weaknesses are common misconfigurations documented in OWASP IoT Top 10. The harness uses only standard pentest tools already in widespread use. We judge the net societal benefit (reproducible evaluation of LLM pentest agents, and a concrete evaluation surface for future defenders) to exceed the incremental misuse risk.

IRB / human subjects. No human subjects are involved. No personal data is collected or processed.

LLM costs and environmental footprint. We report token-estimated cost, actual API billing, and wall-clock time with the experimental results. Energy usage is not claimed unless measured from provider- or host-side telemetry.

10. Conclusion

We presented IoTPentBench, a reproducible benchmark for evaluating LLM penetration testing agents at *network scale* on IoT infrastructure, and MESHSCOUT, a network-native multi-agent harness whose infrastructure-graph guid-

ance and dedicated lateral movement phase keep LLM context tractable while explicitly modelling reachability. Holding the LLM constant (§6) across our pipeline and two ablations, and comparing against Nmap NSE as a non-LLM baseline, the protocol isolates the *architectural* contribution: topology guidance and a dedicated lateral-movement phase can be measured directly through MHR_k and L1–L3 evidence coverage, dimensions that binary flag-capture benchmarks conflate.

Future work. (i) *Multi-model robustness*: extending the architectural comparison to recent frontier and open-weight models to confirm model-independence of the architectural advantage; (ii) *Real hardware*: extending to embedded Zigbee, LoRaWAN, and sub-GHz attack surfaces via SDR tooling; (iii) *Defender’s side*: using IoTPentBench as a benchmark for LLM-assisted IoT defense (detection, patching recommendation).

All code, scenarios, ground truth, and run logs are released at ANONYMIZED_URL under an open license.

References

- [1] A. Happe and J. Cito, “Getting pwn’d by AI: Penetration testing with large language models,” in *Proc. ACM ESEC/FSE*, 2023.
- [2] A. Happe *et al.*, “HackingBuddyGPT: An LLM-driven pentest agent,” 2024. [Online]. Available: <https://github.com/ipa-lab/hackingBuddyGPT>
- [3] G. Deng *et al.*, “PentestGPT: Evaluating and harnessing large language models for automated penetration testing,” in *Proc. 33rd USENIX Security Symp.*, 2024.
- [4] J. Xu *et al.*, “AutoAttacker: A large language model guided system to implement automatic cyber-attacks,” *arXiv:2403.01038*, 2024.
- [5] O. V. Aguinaga, “Nebula: An LLM-powered penetration testing assistant,” *arXiv:2412.00006*, 2024.
- [6] H. He *et al.*, “VulnBot: Autonomous penetration testing for a multi-agent collaborative framework,” *arXiv:2501.13411*, 2025.
- [7] Y. Ginige *et al.*, “AutoPentester: An LLM agent-based framework for automated pentesting,” *arXiv:2510.05605*, 2025.
- [8] X. Shen *et al.*, “PentestAgent: Incorporating LLM agents to automated penetration testing,” in *Proc. 20th ACM Asia Conf. Computer and Communications Security (ASIA CCS)*, 2025. doi: 10.1145/3708821.3733882.
- [9] V. Mayoral-Vilches *et al.*, “CAI: An open, bug bounty-ready cybersecurity AI,” *arXiv:2504.06017*, 2025.
- [10] Anonymous, “Pentest-R1: Towards autonomous penetration testing reasoning optimized via two-stage reinforcement learning,” *arXiv:2508.07382*, 2025.
- [11] Anonymous, “xOffense: An AI-driven autonomous penetration testing framework with offensive knowledge-enhanced LLMs and multi-agent systems,” *arXiv:2509.13021*, 2025.
- [12] T. Abramovich *et al.*, “EnIGMA: Interactive tools substantially assist LM agents in solving CTF challenges,” in *Proc. ICML*, 2025.
- [13] Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-assisted static analysis for detecting security vulnerabilities,” in *Proc. NDSS*, 2025.
- [14] Anonymous, “Context relay for long-running penetration-testing agents,” in *NDSS Workshop on LLM-Assisted Security and Trust Exploration (LAST-X)*, 2026.
- [15] Anonymous, “AWE: A memory-augmented multi-agent framework for autonomous web penetration testing,” in *NDSS Workshop on LLM-Assisted Security and Trust Exploration (LAST-X)*, 2026.
- [16] L. Wang, X. Shi, Z. Li, Y. Jiang, S. Tan, Y. Jiang, J. Cheng, W. Chen, X. Shen, Z. Li, and Y. Chen, “Automated penetration testing with LLM agents and classical planning,” *arXiv:2512.11143*, 2025.
- [17] Anonymous, “Towards automated penetration testing: Introducing LLM benchmark, analysis, and improvements,” *arXiv:2410.17141*, 2024.
- [18] M. Shao *et al.*, “NYU CTF Bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security,” in *Proc. NeurIPS Datasets and Benchmarks Track*, 2024.
- [19] A. K. Zhang *et al.*, “Cybench: A framework for evaluating cybersecurity capabilities and risk of language models,” *arXiv:2408.08926*, 2024.
- [20] M. Bhatt *et al.*, “CyberSecEval 2: A wide-ranging cybersecurity evaluation suite for large language models,” *arXiv:2404.13161*, 2024.
- [21] V. Mayoral-Vilches *et al.*, “CAIBench: A meta-benchmark for evaluating cybersecurity AI agents,” *arXiv:2510.24317*, 2025.
- [22] Ludus Cloud, “Ludus: Cyber range automation on Proxmox.” [Online]. Available: <https://ludus.cloud>
- [23] M3, “GOAD: Game of Active Directory.” [Online]. Available: <https://github.com/Orange-Cyberdefense/GOAD>
- [24] Vulhub contributors, “Vulhub: Pre-built vulnerable Docker environments for learning and reproducing CVEs.” [Online]. Available: <https://github.com/vulhub/vulhub>
- [25] M. Antonakakis *et al.*, “Understanding the Mirai botnet,” in *Proc. 26th USENIX Security Symp.*, 2017.
- [26] W. Turton, “Hackers breach thousands of security cameras, exposing Tesla, jails, hospitals,” *Bloomberg*, Mar. 2021.
- [27] R. Langner, “Stuxnet: Dissecting a cyberwarfare weapon,” *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.
- [28] C. Phillips and L. P. Swiler, “A graph-based system for network-vulnerability analysis,” in *Proc. New Security Paradigms Workshop*, 1998.
- [29] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, “Automated generation and analysis of attack graphs,” in *Proc. IEEE Symp. Security and Privacy*, 2002.
- [30] X. Ou, S. Govindavajhala, and A. W. Appel, “MulVAL: A logic-based network security analyzer,” in *Proc. USENIX Security Symp.*, 2005.
- [31] OWASP Foundation, “OWASP Internet of Things Top 10,” 2024. [Online]. Available: <https://owasp.org/www-project-internet-of-things-top-10/>
- [32] MITRE Corporation, “MITRE ATT&CK for ICS,” 2024. [Online]. Available: <https://attack.mitre.org/matrices/ics/>
- [33] K. Stouffer *et al.*, “Guide to operational technology security,” National Institute of Standards and Technology, NIST SP 800-82 Rev. 3, 2023.
- [34] S. Garcia, A. Parmisano, and M. J. Erquiaga, “IoT-23: A labeled dataset with malicious and benign IoT network traffic,” Stratosphere Laboratory, 2020. [Online]. Available: <https://www.stratosphereips.org/datasets-iot23>
- [35] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, “TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems,” *IEEE Access*, vol. 8, pp. 165130–165150, 2020.
- [36] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, “Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning,” *IEEE Access*, vol. 10, pp. 40281–40306, 2022.
- [37] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, “A large-scale analysis of the security of embedded firmwares,” in *Proc. USENIX Security Symp.*, 2014.
- [38] D. D. Chen, M. Egele, M. Woo, and D. Brumley, “Towards automated dynamic analysis for Linux-based embedded firmware,” in *Proc. NDSS*, 2016.

- [39] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, and Y. Kim, "FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis," in *Proc. Annual Computer Security Applications Conf. (ACSAC)*, 2020.
- [40] OWASP, "IoTGoat: A deliberately insecure IoT firmware," 2024. [Online]. Available: <https://github.com/OWASP/IoTGoat>
- [41] MiniMax, "MiniMax-M2.7: A general-purpose large language model," 2026. [Online]. Available: <https://www.minimax.io/models/text/m27>

LLM Usage Statement

LLMs were used for editorial assistance while preparing this manuscript, and all generated text was inspected by the authors to ensure accuracy, originality, and consistency with the underlying artifacts. LLMs are also part of the studied methodology: MESHSCOUT uses tool-calling LLM agents for network reconnaissance, vulnerability analysis, exploit verification, intrusion attempts, and report generation, as described in §5, §6, and §7.